ARCHITECTURE OPTIONS — NOT A GO-LIVE PLAYBOOK

Majo Bridge Technologies and Events

Hybrid hosting for Lebawi International Academy

LIA physical server + LIA self-hosted cloud VM · never MaJo shared Supabase

Version 1.0 · 10 November 2026, East Africa Time (Africa/Nairobi) · Product: MaJo e-School Bridge (MayaBela)

Hard rule. LIA data must not live on the shared MaJo project `hwkiihonthueadbhcvfi.supabase.co` or on `mayabela.pages.dev` / `www.majobridge.com`. Both machines in this paper are **LIA-owned**: one in the school, one a cloud VM LIA (or MaJo on LIA's behalf) controls.

1. What "hybrid" can mean here

MayaBela is one Flutter app talking to **one** API URL (SUPABASE_URL + anon key). Every desk (attendance, markbook, fees, health, bus GPS) stores rows in the same Postgres table `app_documents`, distinguished only by a `collection` name. Desks do not call each other over HTTP. They **read the same collections and join in the browser** (student id, class name, bus id).

So a hybrid of two machines is possible. A hybrid of two **live writable databases that each own some modules** is not, unless we rewrite the client. The four options below respect that.

Option	Campus physical server	Self-hosted cloud VM	When to pick
A — Campus primary + cloud replica / proxy (recommended)	**Write database, Auth, Edge, staff LAN**	**Public HTTPS, replica, parent path**	**LIA wants data in the building and parents on mobile data**
B — Cloud primary + campus cache	LAN cache, CCTV, print, outbox if WAN dies	Write database (LIA VM, not MaJo)	Campus ISP is the weak link
C — Campus-only sensitive + cloud publish	Health, safeguarding, payroll, HR files	Published grades, attendance, fees due, bus GPS, messages	Some collections must never leave the site
D — Two live writers (do not)	Some modules	Other modules	Split-brain. SIS profile cannot compose a child.

2. How modules read each other today

There is no module-to-module network. The join key is always on the row.

```
Flutter desk (one school JWT)
     |
     +-- DocumentStore / CloudAppStore
     |        HTTPS + JWT
     v
PostgREST  -->  public.app_documents
                (collection, school_id, doc_id, data, updated_at)
                RLS: school_id = jwt schoolId

Edge Functions (login, registry upsert, PIN, mail, AI)
     |  service-role after an application gate
     v
same app_documents + Auth users + Storage buckets
```

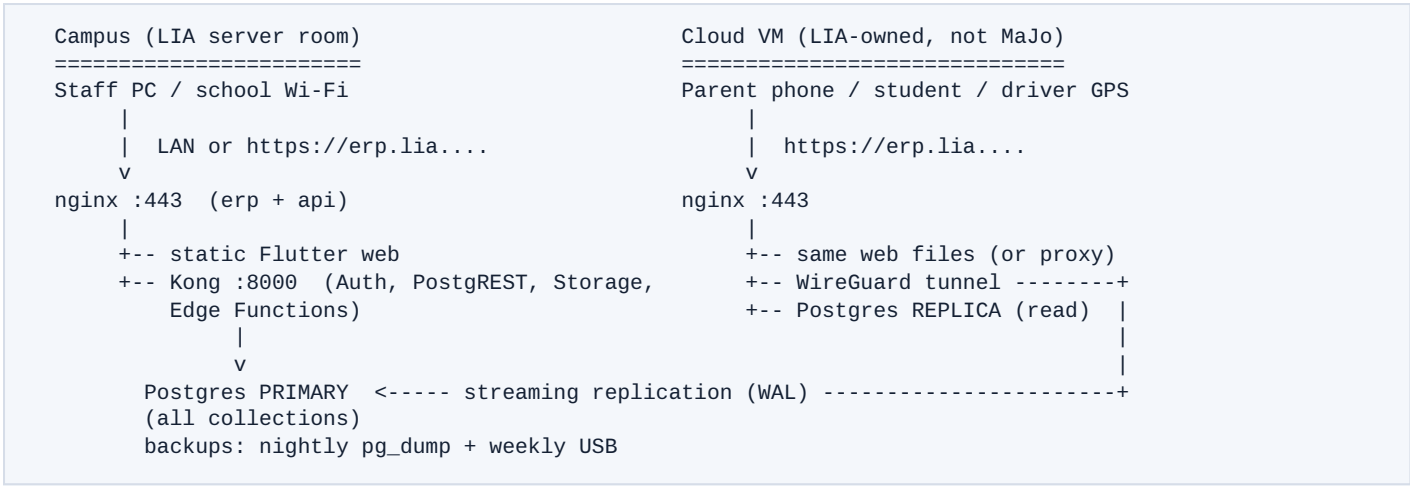
Important joins the client already does (this is why Option D fails):

Desk	Collections it reads	Join key
Students (SIS profile)	`student_registry` + markbook + attendance + fees + health + clubs + documents	student id
Attendance	`attendance_sessions` + `student_registry` + `teacher_registry` (assigned class)	class name / student id
Weighted markbook / report cards	`grade_reports` + students + exam attempts	student id + class
Exam bank	`exam_questions`, `exam_papers`, `exam_attempts` → can push into `grade_reports`	paper / student id
Fees	`fees` + students	student id
Transport / live GPS	`buses` + `driver_registry` + `transport_scans` + `bus_live_positions` + students on that bus	bus id = driver's transportId
Parents	parent link + linked children's students, grades (after publish), attendance, fees, bus	linked student ids
Admissions	`admission_applications` → enrol writes `student_registry`	application → new student id
Inventory / store	`inventory_items`, purchase / issue / transfer requests, `stock_transactions`	item id
LMS	lesson plans + homework + materials + exams + SIS roster	class / student
Payroll / HR	`employee_registry` + `payroll_*` (must stay off parent devices)	staff id
Safeguarding / counseling / IEP	own collections; SIS profile composes a flag, not the case file, for non-cleared roles	student id

Storage is separate: public `school-branding` (login logos) and private `school-files` under `schools/<LIA>/`. Files follow the same school JWT. They should live on the same machine as the write database.

3. Option A — Campus primary, cloud replica / public proxy (recommended)

One LIA database. The physical server is the **only writer**. The cloud VM is the public door (HTTPS, replica, parent phones) and a disaster-recovery copy. This is the smallest change to MayaBela: still one `SUPABASE_URL`.



3.1 Flow — teacher takes attendance

1. Teacher opens `https://erp.lia…` (LAN if on campus, public hostname if at home).
2. Login hits `school-login` Edge on the **campus** stack (via LAN or via the cloud nginx tunnel).
3. Auth mints JWT with `app_metadata.schoolId = LIA`, role = teacher.
4. Attendance desk reads `student_registry` (class list) and writes `attendance_sessions` through PostgREST. RLS checks school + assigned class.
5. WAL ships the new rows to the cloud replica within seconds.

3.2 Flow — parent on mobile data

1. Parent hits the **cloud** hostname (campus public IP may be blocked or unstable from mobile networks).
2. Cloud nginx sends **writes** (messages, link requests, FCM token) through WireGuard to campus Kong.
3. Cloud nginx may serve **reads** from the replica if lag is under a few seconds; otherwise also proxy to campus.
4. Parent JWT can only see linked children. Safeguarding collections stay denied by RLS even though they exist on the replica.

3.3 What to install

- Campus: Ubuntu + Docker self-hosted Supabase (same stack as the LIA on-prem go-live guide) + WireGuard + nightly dump.
- Cloud VM: Ubuntu + nginx + WireGuard + Postgres replica (or a second Docker stack with `hot_standby`). Same TLS names.
- DNS: `erp` and `api` A records → **cloud public IP** (stable). Staff on LAN can also use a split-horizon DNS to the campus LAN IP so a WAN outage does not stop lessons.
- Flutter web/APK: dart-define SUPABASE_URL=`https://api.lia…` and the **LIA anon key**, never the MaJo key.

Why this is first. Modules keep joining in one database. Parents get a public path. Children's rows are not copied to MaJo. Failover is "promote the replica" — a known Postgres operation — not a second product.

4. Option B — Cloud primary, campus as cache / edge

Flip Option A. The self-hosted cloud VM holds the write database. The physical server is the school's always-on LAN box: cached website, local replica, CCTV NVR, print, and the Flutter offline outbox if the ISP dies.

```
Cloud VM (LIA)  =  Postgres PRIMARY + Auth + Edge + public HTTPS
Campus server   =  Flutter web cache + Postgres replica + CCTV +
                   staff devices using local SQLite / CloudAppStore outbox
                   until WAN returns, then flush to cloud primary
```

The product already has `CloudOutboxService` and local stores. Staff can keep taking attendance on campus Wi-Fi during an ISP cut; queued writes upload when the tunnel is back. Parents cannot work during a cloud-VM outage (they were never on the LAN).

Pick B if Ethio/campus last-mile is worse than a proper VM in a licensed local data centre. **Do not pick B** if LIA's TOR says the system of record must sit in the building.

5. Option C — Campus-only sensitive collections, cloud publish

Two LIA databases, but **not** two writers for the same child. Campus is the system of record. A publish worker copies only allowed collections to the cloud VM that parents hit.

Stays on campus only	Published to cloud (parent/student/driver)
`health_records`, `counseling_records`, `iep_plans`, `safeguarding_cases`, `medication_stock`, payroll, HR files, staff documents	Published `grade_reports`, attendance summaries, homework, announcements, fees due (not the finance ledger), bus GPS, messages, branding

```
Staff desks  -->  Campus PRIMARY (all collections)
                        |
                        |  publish worker (allow-list, no medical/safeguarding/payroll)
                        v
Parent / student / driver  -->  Cloud VM (subset replica)
```

5.1 Flow — publish grades

1. Teacher writes marks on campus. They stay campus-only until academic lead approves/publishes.
2. Publish sets a flag on `grade_reports`. Worker copies those rows (and the linked student display name, not the medical flag) to the cloud DB.
3. Parent app uses a **second** API URL (parent build) that only knows the cloud VM.

> **Cost.** This needs a new worker, two Flutter builds (staff vs parent), and a rule that SIS-on-cloud must not try to open health collections. Use it only if LIA legal/INSA says those collections must not be replicated off-site at all — even to an LIA VM.

6. Option D — Split modules across two live writers (do not)

Example: "academics on cloud, finance on campus". The SIS profile, report cards, at-risk rules, and parent app all join those collections in memory. Two writers means two student ids, delayed fees, and RLS that cannot see the other school-half. We would have to invent a sync bus between modules. That is a second product.

7. End-to-end flows (Option A)

7.1 Login

```
Device --HTTPS--> nginx (cloud or campus split-DNS)
                    --> Kong --> school-login (rate-limited)
                    --> bcrypt verify (schoolId+username+role)
                    --> Auth JWT (schoolId=LIA, role)
Device stores JWT. Later calls send it. No JWT on the login page (logo is public branding).
```

7.2 Ordinary classroom write

```
Teacher JWT --> PostgREST INSERT attendance_sessions
                  RLS: school_id = LIA AND assigned class
                  row lands on PRIMARY
                  replica receives WAL
At-risk desk later READs attendance_sessions + grade_reports + student_registry
                  (same DB, same JWT, join in Flutter)
```

7.3 Privileged registry write

```
Registrar JWT --> Edge school-upsert-registry
                  gate (admissions / student row)
                  service-role upsert on PRIMARY
                  student_registry row appears
Fees, transport, markbook can now join that student id
                  — they do not wait on a second module API
```

7.4 Driver GPS

```
Driver phone --> cloud nginx --> tunnel --> PRIMARY
                 bus_live_positions (transportId == busId)
Parent read  --> replica or tunnel --> same collection, linked child only
```

7.5 WAN down at campus (Option A + split-horizon)

```
Staff on school Wi-Fi --> LAN_IP nginx --> PRIMARY   (lessons continue)
Parents on mobile     --> cloud nginx  --> tunnel X  (read replica until lag policy;
                                                      writes queue or show “campus unreachable”)

USB / pg_dump still runs on campus
```

8. Comparison

	A Campus primary	B Cloud primary	C Publish subset	D Two writers
Matches current Flutter (one URL)	Yes	Yes	No — two builds	No
Modules join without a new bus	Yes	Yes	Staff yes / parent limited	No
Record in the building	Yes	No (replica only)	Yes for sensitive	Confused
Parents on mobile data	Yes (cloud door)	Yes	Yes (subset)	Fragile
Lessons if ISP dies	Yes if split-DNS to LAN	Yes via local outbox	Staff yes	Unknown

	A Campus primary	B Cloud primary	C Publish subset	D Two writers
INSA / TOR “not MaJo cloud”	Yes	Yes if VM is LIA’s	Yes	Yes, but unsafe
Engineering size	Ops (WireGuard + replica)	Ops + outbox tuning	New worker + two apps	Rewrite

9. Recommendation

1. **Implement Option A.** Campus self-hosted Supabase as primary. Cloud VM as public nginx + WireGuard + replica. One LIA Flutter build.
2. Use **split-horizon DNS** so teachers on campus Wi-Fi never depend on the ISP for the API.
3. Keep Option C in reserve only for health/safeguarding/payroll if LIA legal forbids off-site replicas of those collections — then exclude them from WAL (publication set) rather than inventing a second SIS.
4. Do not point anything at MaJo’s shared Supabase. Do not fail over LIA onto `www.majobridge.com`.

Next physical step (not in this paper): the existing LIA on-prem go-live guide for Ubuntu + Docker + nginx + Certbot on the campus box, then a second identical VM with replica settings and WireGuard. Confirm LIA’s public IP, domain, and which licensed local cloud (or LIA VPS) they will rent.

MB-LIA-HYB-001 · MaJo e-School Bridge · 10 November 2026, Africa/Nairobi · Companion: LIA on-prem go-live guide (campus stack).
This file is options and flows, not a certificate and not a live cutover.